

```

In [15]: # =====
# STEAM REVIEW & ENGAGEMENT ANALYSIS
# Analysis of 6.4 Million Steam User Reviews
# =====

# =====
# 1. LIBRARIES AND SETTINGS
# =====

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from IPython.display import display, Markdown

sns.set_theme(style="whitegrid")

plt.rcParams["figure.figsize"] = (12, 7)
plt.rcParams["font.size"] = 11
plt.rcParams["axes.titleweight"] = "bold"
plt.rcParams["axes.titlesize"] = 15

pd.set_option("display.max_columns", 20)
pd.set_option("display.float_format", lambda x: f"{x:,.2f}")

display(Markdown("""
# Steam Review & Engagement Analysis

### Analysis of 6.4 Million Steam User Reviews

**Tools:** Python, Pandas, NumPy, Matplotlib, Seaborn
**Focus:** Data quality, approval rates, review concentration, and helpfu
"""))

# =====
# 2. LOAD DATA
# =====

file_path = r"C:\Users\שלי\מחשב\Downloads\archive (3)\dataset.csv"

df = pd.read_csv(
    file_path,
    usecols=[
        "app_id",
        "app_name",
        "review_score",
        "review_votes"
    ],
    low_memory=False
)

```

```

print("Dataset loaded successfully.")
print(f"Total records: {len(df):,}")
print(f"Columns used: {len(df.columns)}")
print(
    f"Memory usage: "
    f"{df.memory_usage(deep=True).sum() / 1024**2:,.1f} MB"
)

display(df.head(10))

# =====
# 3. DATA QUALITY AUDIT
# =====

print("\n" + "=" * 55)
print("DATA QUALITY AUDIT")
print("=" * 55)

df["review_score"] = pd.to_numeric(
    df["review_score"],
    errors="coerce"
)

df["review_votes"] = pd.to_numeric(
    df["review_votes"],
    errors="coerce"
)

# Missing values
missing_report = pd.DataFrame({
    "missing_values": df.isna().sum(),
    "missing_percentage": df.isna().mean() * 100
})

print("\nMissing Values")

display(
    missing_report.round(2)
)

# Zero values
zero_counts = (
    df[
        [
            "app_id",
            "review_score",
            "review_votes"
        ]
    ] == 0
).sum()

zero_report = pd.DataFrame({
    "zero_values": zero_counts,

```

```

    "zero_percentage": zero_counts / len(df) * 100
})

print("\nZero Values")

display(
    zero_report.round(2)
)

# Unique values
score_values = sorted(
    df["review_score"]
    .dropna()
    .unique()
    .tolist()
)

vote_values = sorted(
    df["review_votes"]
    .dropna()
    .unique()
    .tolist()
)

print(f"\nUnique review_score values: {score_values}")
print(f"Unique review_votes values: {vote_values}")

if not set(score_values).issubset({-1, 1}):
    raise ValueError(
        "Unexpected values found in review_score."
    )

if not set(vote_values).issubset({0, 1}):
    raise ValueError(
        "Unexpected values found in review_votes."
    )

print("\nValue validation completed successfully.")
print("-1 = Negative review")
print(" 1 = Positive review")
print(" 0 = Review not marked as Helpful")
print(" 1 = Review marked as Helpful")

# Potential duplicates
potential_duplicates = df.duplicated().sum()

print(
    f"\nPotential duplicate rows based on loaded columns: "
    f"{potential_duplicates:,}"
)

print(

```

```

        "These rows were not removed because the dataset "
        "does not contain a unique review identifier."
    )

# Descriptive statistics
print("\nReview-Level Descriptive Statistics")

display(
    df[
        [
            "review_score",
            "review_votes"
        ]
    ]
    .describe()
    .T
    .round(2)
)

# =====
# 4. DATA CLEANING
# =====

original_rows = len(df)

df_clean = df.dropna(
    subset=[
        "app_id",
        "app_name",
        "review_score",
        "review_votes"
    ]
).copy()

df_clean = df_clean[
    df_clean["review_score"].isin([-1, 1])
    & df_clean["review_votes"].isin([0, 1])
].copy()

df_clean["app_name"] = (
    df_clean["app_name"]
    .astype("string")
    .str.strip()
)

df_clean = df_clean[
    df_clean["app_name"].ne("")
].copy()

df_clean["is_positive"] = (
    df_clean["review_score"] == 1
).astype(int)

```

```

df_clean["is_helpful"] = (
    df_clean["review_votes"] == 1
).astype(int)

removed_rows = original_rows - len(df_clean)

positive_rate = (
    df_clean["is_positive"].mean()
    * 100
)

helpful_rate = (
    df_clean["is_helpful"].mean()
    * 100
)

not_helpful_rate = (
    100 - helpful_rate
)

print("\n" + "=" * 55)
print("DATA CLEANING SUMMARY")
print("=" * 55)

print(f"Original records: {original_rows:,}")
print(f"Removed records: {removed_rows:,}")
print(f"Clean records: {len(df_clean):,}")
print(f"Positive reviews: {positive_rate:.2f}%")
print(f"Reviews marked Helpful: {helpful_rate:.2f}%")
print(f"Reviews not marked Helpful: {not_helpful_rate:.2f}%")

display(Markdown(f"""
### Data Cleaning Summary

- The original dataset contained **{original_rows:,} individual reviews**
- **{removed_rows:,} records** were removed because essential fields were
- The cleaned dataset contains **{len(df_clean):,} valid reviews**.
- Zero values in `review_votes` were retained because they represent vali
- The written review text was not loaded because the current project does
"""))

# =====
# 5. REVIEW-LEVEL DATA PREVIEW
# =====

display(Markdown("""
## Review-Level Data Preview

Before aggregation, each row represents one individual Steam review.

The same game may appear in many rows because each review is stored separ
"""))

```

```

display(
    df_clean[
        [
            "app_id",
            "app_name",
            "review_score",
            "review_votes",
            "is_positive",
            "is_helpful"
        ]
    ].head(10)
)

# =====
# 6. GAME-LEVEL AGGREGATION
# =====

game_stats = (
    df_clean
    .groupby(
        "app_name",
        as_index=False
    )
    .agg(
        total_reviews=(
            "review_score",
            "size"
        ),
        positive_reviews=(
            "is_positive",
            "sum"
        ),
        helpful_reviews=(
            "is_helpful",
            "sum"
        )
    )
)

game_stats["approval_rate"] = (
    game_stats["positive_reviews"]
    / game_stats["total_reviews"]
    * 100
)

game_stats["helpful_review_rate"] = (
    game_stats["helpful_reviews"]
    / game_stats["total_reviews"]
    * 100
)

game_stats = game_stats[

```

```

        [
            "app_name",
            "total_reviews",
            "positive_reviews",
            "approval_rate",
            "helpful_reviews",
            "helpful_review_rate"
        ]
    ]

print("\n" + "=" * 55)
print("REVIEW-LEVEL TO GAME-LEVEL AGGREGATION")
print("=" * 55)

print(f"Clean individual reviews: {len(df_clean):,}")
print(f"Unique game names: {df_clean['app_name'].nunique():,}")
print(f"Rows in game_stats: {len(game_stats):,}")

display(Markdown(f"""
## From Individual Reviews to Game-Level Statistics

The cleaned dataset contains **{len(df_clean):,} individual review rows**

The reviews were grouped by game name using `groupby("app_name")`.

This produced **{len(game_stats):,} game-level summaries**.

**{len(df_clean):,} individual reviews → {len(game_stats):,} game-level r

Each row in `game_stats` represents one game and contains:

- Total reviews
- Positive reviews
- Approval rate
- Reviews marked as Helpful
- Helpful-review rate
"""))

display(Markdown("""
## Game-Level Table Preview

After aggregation, each row represents one game.
"""))

display(
    game_stats.head(10)
)

print("\nGame-Level Descriptive Statistics")

display(
    game_stats[
        [
            "total_reviews",
            "approval_rate",

```

```

        "helpful_reviews",
        "helpful_review_rate"
    ]
]
.describe()
.round(2)
)

# =====
# Q1: WHAT IS THE OVERALL REVIEW SENTIMENT?
# =====

sentiment_counts = (
    df_clean["is_positive"]
    .value_counts()
    .reindex(
        [1, 0],
        fill_value=0
    )
)

plt.figure(figsize=(8, 6))

plt.pie(
    sentiment_counts.values,
    labels=[
        "Positive",
        "Negative"
    ],
    autopct="%1.1f%%",
    colors=[
        "#2A9D8F",
        "#E76F51"
    ],
    startangle=90,
    explode=(0, 0.05),
    wedgeprops={
        "edgecolor": "white",
        "linewidth": 2
    }
)

plt.title(
    "Q1: Overall Review Sentiment"
)

plt.show()

display(Markdown(f"""
### Q1 Findings

- Positive reviews represented {positive_rate:.1f}% of the analyzed d
- Overall review sentiment was strongly positive.
- Games performing substantially below this benchmark may require deeper
"""))

```

```

# =====
# Q2: HOW CONCENTRATED IS REVIEW ACTIVITY?
# =====

pareto_data = (
    game_stats
    .sort_values(
        "total_reviews",
        ascending=False
    )
    .copy()
)

pareto_data["cumulative_review_share"] = (
    pareto_data["total_reviews"]
    .cumsum()
    / pareto_data["total_reviews"].sum()
    * 100
)

pareto_data["catalog_share"] = (
    np.arange(
        1,
        len(pareto_data) + 1
    )
    / len(pareto_data)
    * 100
)

top_five_count = max(
    1,
    int(
        np.ceil(
            len(pareto_data) * 0.05
        )
    )
)

top_five_share = (
    pareto_data
    .head(top_five_count)["total_reviews"]
    .sum()
    / pareto_data["total_reviews"].sum()
    * 100
)

plt.figure()

plt.plot(
    pareto_data["catalog_share"],
    pareto_data["cumulative_review_share"],
    color="#264653",
    linewidth=3
)

```

```

plt.axvline(
    5,
    color="#F4A261",
    linestyle="--",
    label="Top 5% of Games"
)

plt.axhline(
    80,
    color="#E76F51",
    linestyle="--",
    label="80% of Reviews"
)

plt.title(
    "Q2: Review Concentration by Game"
)

plt.xlabel("Share of Game Catalog (%)")
plt.ylabel("Cumulative Share of Reviews (%)")

plt.legend()
plt.tight_layout()
plt.show()

display(Markdown(f"""
### Q2 Findings

- The top **5% of games generated {top_five_share:.1f}% of all reviews**.
- Review activity was highly concentrated among a relatively small share
- Visibility and community activity were not distributed evenly across th
"""))

# =====
# Q3: WHICH MAJOR GAMES HAD THE MOST DIVIDED SENTIMENT?
# =====

major_games = game_stats[
    game_stats["total_reviews"] > 20_000
].copy()

major_games["distance_from_50"] = abs(
    major_games["approval_rate"] - 50
)

polarized_games = (
    major_games
    .nsmallest(
        10,
        "distance_from_50"
    )
)

plt.figure()

```

```

sns.barplot(
    data=polarized_games,
    x="approval_rate",
    y="app_name",
    color="#6C8EBF"
)

plt.axvline(
    50,
    color="black",
    linestyle="--",
    label="50% Approval"
)

plt.xlim(0, 100)

plt.title(
    "Q3: Most Polarized Major Games"
)

plt.xlabel("Approval Rate (%)")
plt.ylabel("Game")

plt.legend()
plt.tight_layout()
plt.show()

display(
    polarized_games[
        [
            "app_name",
            "total_reviews",
            "approval_rate"
        ]
    ]
)

display(Markdown("""
### Q3 Findings

- These games recorded approval rates closest to a 50/50 split.
- The results indicate divided sentiment among games with more than 20,00
- Review-text analysis would be required to determine the reasons behind
"""))

# =====
# Q4: HOW ARE HIGH-VOLUME GAMES POSITIONED BY APPROVAL?
# =====

portfolio_data = game_stats[
    game_stats["total_reviews"] > 5_000
].copy()

lower_approval_games = (
    portfolio_data[
        portfolio_data["approval_rate"] < 80

```

```

    ]
    .nlargest(
        10,
        "total_reviews"
    )
)

lower_approval_count = len(
    portfolio_data[
        portfolio_data["approval_rate"] < 80
    ]
)

plt.figure()

sns.scatterplot(
    data=portfolio_data,
    x="approval_rate",
    y="total_reviews",
    hue="approval_rate",
    palette="RdYlGn",
    s=100,
    alpha=0.75,
    legend=False
)

plt.yscale("log")

plt.axvline(
    80,
    color="black",
    linestyle="--"
)

plt.title(
    "Q4: Approval Rate vs. Review Volume"
)

plt.xlabel("Approval Rate (%)")
plt.ylabel("Total Reviews – Log Scale")

plt.tight_layout()
plt.show()

display(
    lower_approval_games[
        [
            "app_name",
            "total_reviews",
            "approval_rate",
            "helpful_review_rate"
        ]
    ]
)

display(Markdown(f"""

```

Q4 Findings

- ****{lower_approval_count:,} high-volume games**** recorded approval rates
- These games combined substantial review activity with comparatively weak approval rates
- They may be useful candidates for deeper product and review analysis.

```
""")
```

```
# =====  
# Q5: ARE NEGATIVE REVIEWS MORE LIKELY TO BE HELPFUL?  
# =====
```

```
df_clean["sentiment"] = np.where(  
    df_clean["is_positive"] == 1,  
    "Positive",  
    "Negative"  
)
```

```
helpful_by_sentiment = (  
    df_clean  
    .groupby(  
        "sentiment",  
        as_index=False  
    )  
    .agg(  
        total_reviews=(  
            "is_helpful",  
            "size"  
        ),  
  
        helpful_review_rate=(  
            "is_helpful",  
            "mean"  
        )  
    )  
)
```

```
helpful_by_sentiment["helpful_review_rate"] *= 100
```

```
negative_helpful_rate = helpful_by_sentiment.loc[  
    helpful_by_sentiment["sentiment"] == "Negative",  
    "helpful_review_rate"  
].iloc[0]
```

```
positive_helpful_rate = helpful_by_sentiment.loc[  
    helpful_by_sentiment["sentiment"] == "Positive",  
    "helpful_review_rate"  
].iloc[0]
```

```
helpful_ratio = (  
    negative_helpful_rate  
    / positive_helpful_rate  
)
```

```

plt.figure()

sns.barplot(
    data=helpful_by_sentiment,
    x="sentiment",
    y="helpful_review_rate",
    order=[
        "Negative",
        "Positive"
    ],
    hue="sentiment",
    palette={
        "Negative": "#E76F51",
        "Positive": "#2A9D8F"
    },
    legend=False
)

plt.title(
    "Q5: Helpful-Review Rate by Sentiment"
)

plt.xlabel("Review Sentiment")
plt.ylabel("Reviews Marked Helpful (%)")

plt.tight_layout()
plt.show()

display(
    helpful_by_sentiment
)

display(Markdown(f"""
### Q5 Findings

- **{negative_helpful_rate:.1f}% of negative reviews** were marked as Hel
- **{positive_helpful_rate:.1f}% of positive reviews** were marked as Hel
- Negative reviews were approximately **{helpful_ratio:.1f} times more li
- Critical feedback may therefore receive greater community visibility.
"""))

# =====
# Q6: HOW DOES APPROVAL VARY BY REVIEW-VOLUME TIER?
# =====

game_stats["review_volume_tier"] = pd.cut(
    game_stats["total_reviews"],
    bins=[
        0,
        1_000,
        10_000,
        np.inf
    ],
    labels=[
        "Low Volume (<1K)",
        "Medium Volume (1K-10K)",
        "High Volume (>10K)"
    ]
)

```

```

    ],
    include_lowest=True
)

tier_summary = (
    game_stats
    .groupby(
        "review_volume_tier",
        observed=True,
        as_index=False
    )
    .agg(
        game_count=(
            "app_name",
            "count"
        ),
        average_approval=(
            "approval_rate",
            "mean"
        )
    )
)

low_volume_approval = (
    tier_summary.iloc[0][
        "average_approval"
    ]
)

high_volume_approval = (
    tier_summary.iloc[-1][
        "average_approval"
    ]
)

approval_difference = (
    high_volume_approval
    - low_volume_approval
)

plt.figure()

sns.barplot(
    data=tier_summary,
    x="review_volume_tier",
    y="average_approval",
    hue="review_volume_tier",
    palette="viridis",
    legend=False
)

plt.ylim(0, 100)

plt.title(

```

```

    "Q6: Average Approval by Review-Volume Tier"
)

plt.xlabel("Review-Volume Tier")
plt.ylabel("Average Approval Rate (%)")

plt.tight_layout()
plt.show()

display(
    tier_summary
)

display(Markdown(f"""
### Q6 Findings

- Average approval increased from {low_volume_approval:.1f}% in the 1
- The difference was approximately {approval_difference:.1f} percentage
- This is a descriptive relationship and does not prove causality.
"""))

# =====
# Q7: WHICH GAMES HAD THE HIGHEST HELPFUL-REVIEW RATE?
# =====

top_helpful_games = (
    game_stats[
        game_stats["total_reviews"] > 10_000
    ]
    .nlargest(
        10,
        "helpful_review_rate"
    )
)

plt.figure()

sns.barplot(
    data=top_helpful_games,
    x="helpful_review_rate",
    y="app_name",
    color="#7B2CBF"
)

plt.title(
    "Q7: Top Games by Helpful-Review Rate"
)

plt.xlabel("Reviews Marked Helpful (%)")
plt.ylabel("Game")

plt.tight_layout()
plt.show()

display(

```

```

top_helpful_games[
    [
        "app_name",
        "total_reviews",
        "approval_rate",
        "helpful_review_rate"
    ]
]
)

top_helpful_game = (
    top_helpful_games.iloc[0]
)

display(Markdown(f"""
### Q7 Findings

- **{top_helpful_game['app_name']}** had the highest helpful-review rate
- Approximately **{top_helpful_game['helpful_review_rate']:.1f}%** of its
- This indicates relatively strong interaction with its review content.
"""))

# =====
# Q8: IS REVIEW VOLUME RELATED TO APPROVAL?
# =====

correlation_data = game_stats[
    game_stats["total_reviews"] > 1_000
].copy()

volume_approval_corr = (
    correlation_data["total_reviews"]
    .corr(
        correlation_data["approval_rate"]
    )
)

plt.figure()

sns.regplot(
    data=correlation_data,
    x="total_reviews",
    y="approval_rate",
    scatter_kws={
        "alpha": 0.20,
        "color": "#2A9D8F"
    },
    line_kws={
        "color": "#E76F51"
    }
)

plt.xscale("log")

plt.title(

```

```

    "Q8: Review Volume vs. Approval Rate "
    f"(r = {volume_approval_corr:.3f})"
)

plt.xlabel("Total Reviews – Log Scale")
plt.ylabel("Approval Rate (%)")

plt.tight_layout()
plt.show()

display(Markdown(f"""
### Q8 Findings

- Pearson correlation was {volume_approval_corr:.3f}.
- The relationship between review volume and approval rate was very weak.
- Review volume alone is not a reliable indicator of user satisfaction.
- The logarithmic scale was used only to improve chart readability.
"""))

# =====
# Q9: IS APPROVAL RELATED TO HELPFUL-REVIEW RATE?
# =====

approval_helpful_corr = (
    correlation_data["approval_rate"]
    .corr(
        correlation_data["helpful_review_rate"]
    )
)

plt.figure()

sns.regplot(
    data=correlation_data,
    x="approval_rate",
    y="helpful_review_rate",
    scatter_kws={
        "alpha": 0.25,
        "color": "#7B8DB5"
    },
    line_kws={
        "color": "#F4A261"
    }
)

plt.title(
    "Q9: Approval Rate vs. Helpful-Review Rate "
    f"(r = {approval_helpful_corr:.3f})"
)

plt.xlabel("Approval Rate (%)")
plt.ylabel("Reviews Marked Helpful (%)")

plt.tight_layout()
plt.show()

```

```

display(Markdown(f"""
### Q9 Findings

- Pearson correlation was {approval_helpful_corr:.3f}.
- The result shows a weak negative relationship.
- Games with higher approval rates tended to have slightly lower helpful-
- This relationship does not prove causality.
"""))

# =====
# Q10: HOW CONCENTRATED IS HELPFUL-REVIEW ACTIVITY?
# =====

top_one_count = max(
    1,
    int(
        np.ceil(
            len(game_stats) * 0.01
        )
    )
)

top_one_games = game_stats.nlargest(
    top_one_count,
    "helpful_reviews"
)

top_one_share = (
    top_one_games["helpful_reviews"].sum()
    / game_stats["helpful_reviews"].sum()
    * 100
)

helpful_share = pd.DataFrame({
    "group": [
        "Top 1% of Games",
        "Rest of Games"
    ],
    "share": [
        top_one_share,
        100 - top_one_share
    ]
})

plt.figure(figsize=(9, 6))

sns.barplot(
    data=helpful_share,
    x="group",
    y="share",
    hue="group",
    palette={
        "Top 1% of Games": "#E76F51",
        "Rest of Games": "#7B2CBF"
    }
)

```

```

    },
    legend=False
)

plt.title(
    "Q10: Helpful-Review Concentration"
)

plt.xlabel("")
plt.ylabel("Share of Helpful Reviews (%)")

plt.tight_layout()
plt.show()

display(
    helpful_share
)

display(Markdown(f"""
### Q10 Findings

- The top **1% of games accounted for {top_one_share:.1f}% of all reviews
- Helpful-review activity was highly concentrated among a small share of
- This result refers to review interaction, not sales or total player act
"""))

# =====
# FINAL EXECUTIVE SUMMARY
# =====

display(Markdown(f"""
# Executive Summary

- The original dataset contained **{original_rows:,} Steam reviews**.
- After cleaning, **{len(df_clean):,} valid reviews** remained.
- The reviews were aggregated into **{len(game_stats):,} game-level summa
- Positive reviews represented **{positive_rate:.1f}%** of the dataset.
- **{not_helpful_rate:.1f}%** of reviews were not marked as Helpful.
- The top **5% of games generated {top_five_share:.1f}% of all reviews**.
- Negative reviews were approximately **{helpful_ratio:.1f} times more li
- Average approval increased by approximately **{approval_difference:.1f}
- Review volume had a correlation of **{volume_approval_corr:.3f}** with
- Approval rate had a correlation of **{approval_helpful_corr:.3f}** with
- The top **1% of games accounted for {top_one_share:.1f}% of all Helpful

## Business Interpretation

Review activity and Helpful engagement were concentrated among a relative

Overall sentiment was positive, but review volume alone was not a reliabl

The findings describe relationships within the dataset and do not prove c
"""))

print("\n" + "=" * 55)

```

```
print("STEAM REVIEW ANALYSIS COMPLETED SUCCESSFULLY")
print("=" * 55)
```

Steam Review & Engagement Analysis

Analysis of 6.4 Million Steam User Reviews

Tools: Python, Pandas, NumPy, Matplotlib, Seaborn

Focus: Data quality, approval rates, review concentration, and helpful-review engagement

Dataset loaded successfully.

Total records: 6,417,106

Columns used: 4

Memory usage: 552.5 MB

	app_id	app_name	review_score	review_votes
0	10	Counter-Strike	1	0
1	10	Counter-Strike	1	1
2	10	Counter-Strike	1	0
3	10	Counter-Strike	1	0
4	10	Counter-Strike	1	1
5	10	Counter-Strike	1	1
6	10	Counter-Strike	1	1
7	10	Counter-Strike	1	1
8	10	Counter-Strike	1	1
9	10	Counter-Strike	1	0

DATA QUALITY AUDIT

Missing Values

	missing_values	missing_percentage
app_id	0	0.00
app_name	183234	2.86
review_score	0	0.00
review_votes	0	0.00

Zero Values

	zero_values	zero_percentage
app_id	0	0.00
review_score	0	0.00
review_votes	5472222	85.28

Unique review_score values: [-1, 1]

Unique review_votes values: [0, 1]

Value validation completed successfully.

-1 = Negative review

1 = Positive review

0 = Review not marked as Helpful

1 = Review marked as Helpful

Potential duplicate rows based on loaded columns: 6,381,140

These rows were not removed because the dataset does not contain a unique review identifier.

Review-Level Descriptive Statistics

	count	mean	std	min	25%	50%	75%	max
review_score	6,417,106.00	0.64	0.77	-1.00	1.00	1.00	1.00	1.00
review_votes	6,417,106.00	0.15	0.35	0.00	0.00	0.00	0.00	1.00

DATA CLEANING SUMMARY

Original records: 6,417,106

Removed records: 183,234

Clean records: 6,233,872

Positive reviews: 82.33%

Reviews marked Helpful: 14.66%

Reviews not marked Helpful: 85.34%

Data Cleaning Summary

- The original dataset contained **6,417,106 individual reviews**.
- **183,234 records** were removed because essential fields were missing or invalid.
- The cleaned dataset contains **6,233,872 valid reviews**.
- Zero values in `review_votes` were retained because they represent valid reviews that were not marked as Helpful.
- The written review text was not loaded because the current project does not perform text analysis.

Review-Level Data Preview

Before aggregation, each row represents one individual Steam review.

The same game may appear in many rows because each review is stored separately.

	app_id	app_name	review_score	review_votes	is_positive	is_helpful
0	10	Counter-Strike	1	0	1	0
1	10	Counter-Strike	1	1	1	1
2	10	Counter-Strike	1	0	1	0
3	10	Counter-Strike	1	0	1	0
4	10	Counter-Strike	1	1	1	1
5	10	Counter-Strike	1	1	1	1
6	10	Counter-Strike	1	1	1	1
7	10	Counter-Strike	1	1	1	1
8	10	Counter-Strike	1	1	1	1
9	10	Counter-Strike	1	0	1	0

```
=====
REVIEW-LEVEL TO GAME-LEVEL AGGREGATION
=====
Clean individual reviews: 6,233,872
Unique game names: 9,363
Rows in game_stats: 9,363
```

From Individual Reviews to Game-Level Statistics

The cleaned dataset contains **6,233,872 individual review rows**.

The reviews were grouped by game name using `groupby("app_name")`.

This produced **9,363 game-level summaries**.

6,233,872 individual reviews → 9,363 game-level rows

Each row in `game_stats` represents one game and contains:

- Total reviews
- Positive reviews
- Approval rate
- Reviews marked as Helpful
- Helpful-review rate

Game-Level Table Preview

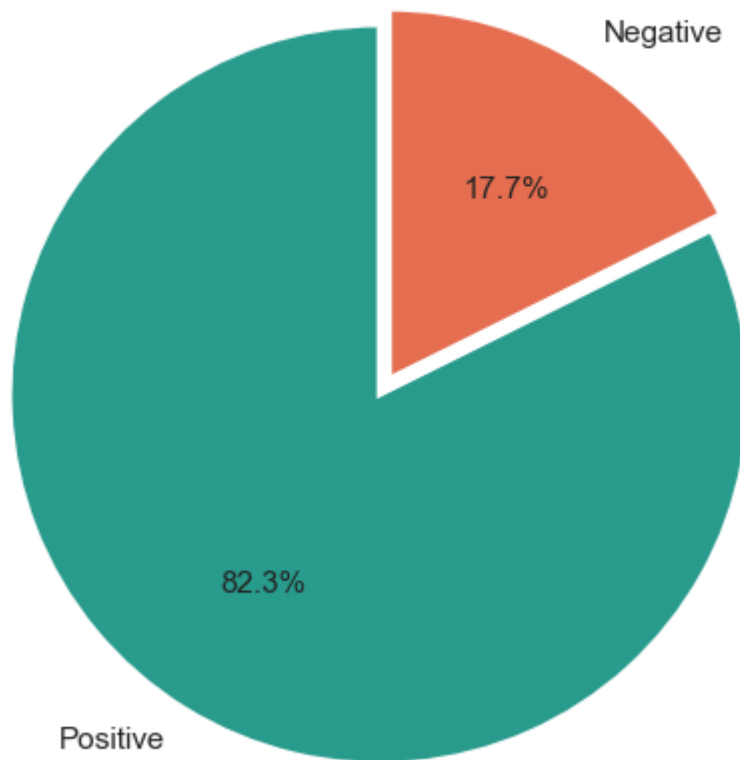
After aggregation, each row represents one game.

	app_name	total_reviews	positive_reviews	approval_rate	helpful_reviews	help
0	! That Bastard Is Trying To Steal Our Gold !	26	8	30.77	14	
1	!"We Are The Dwarves"!	103	79	76.70	35	
2	"Glow Ball" - The billiard puzzle game	27	5	18.52	13	
3	#SelfieTennis	63	47	74.60	20	
4	#killallzombies	7	5	71.43	1	
5	\$1 Ride	26	9	34.62	12	
6	'n Verlore Verstand	19	11	57.89	11	
7	.EXE	6	5	83.33	3	
8	001 Game Creator	39	34	87.18	15	
9	ORBITALIS	172	129	75.00	29	

Game-Level Descriptive Statistics

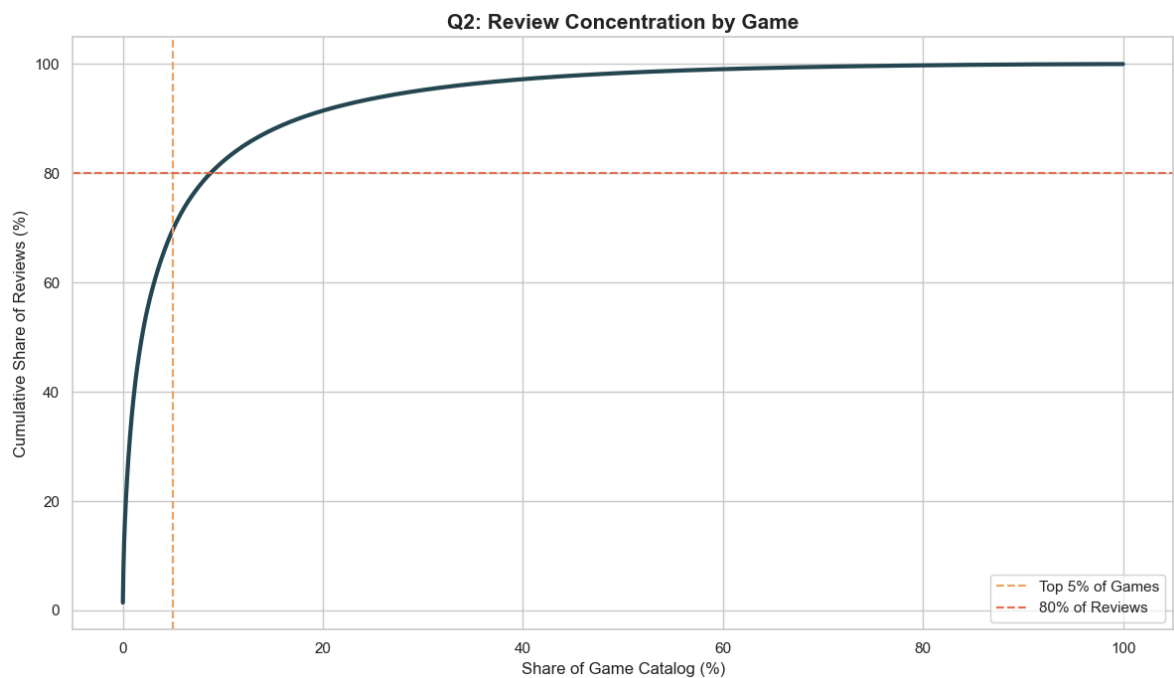
	total_reviews	approval_rate	helpful_reviews	helpful_review_rate
count	9,363.00	9,363.00	9,363.00	9,363.00
mean	665.80	72.72	97.63	27.71
std	3,187.72	21.44	449.76	20.37
min	1.00	0.00	0.00	0.00
25%	18.00	60.00	5.00	12.69
50%	58.00	77.78	14.00	23.08
75%	244.00	89.41	46.00	37.50
max	88,973.00	100.00	14,462.00	100.00

Q1: Overall Review Sentiment



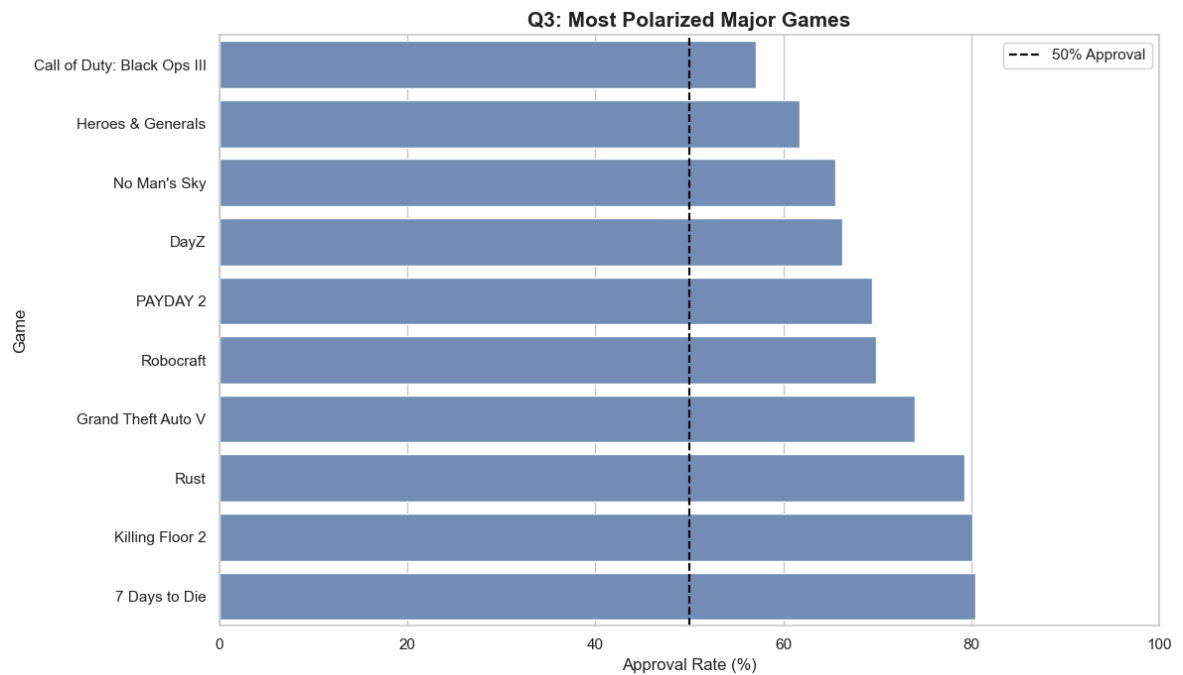
Q1 Findings

- Positive reviews represented **82.3%** of the analyzed dataset.
- Overall review sentiment was strongly positive.
- Games performing substantially below this benchmark may require deeper product or technical analysis.



Q2 Findings

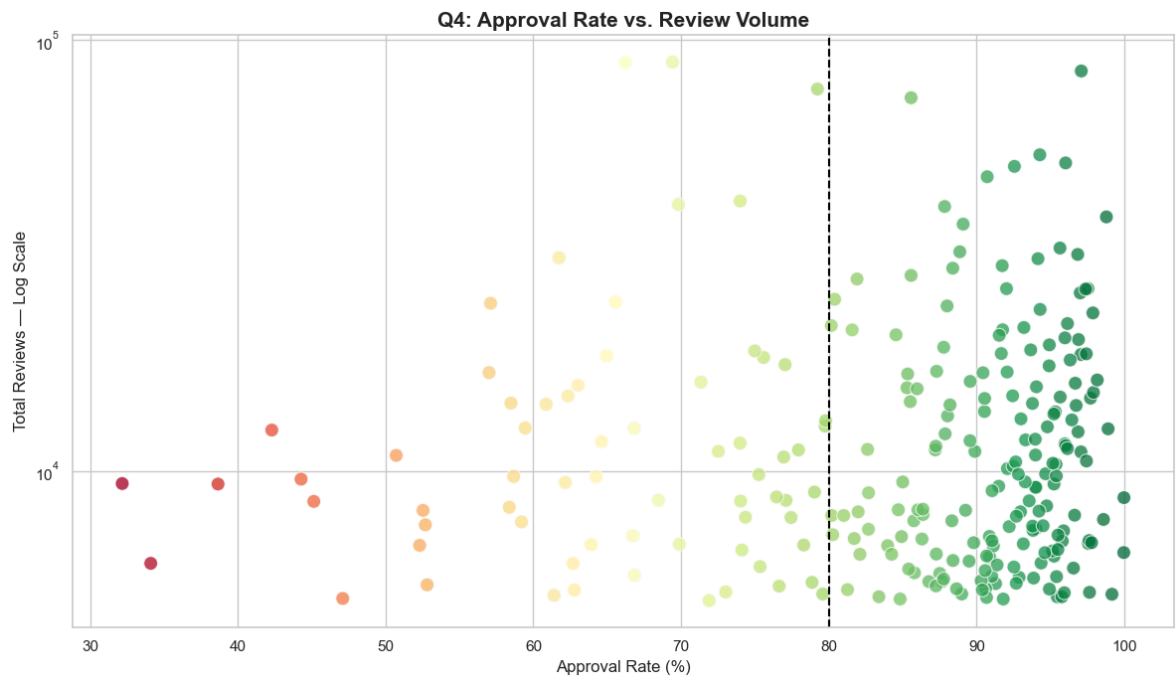
- The top **5% of games generated 69.7% of all reviews.**
- Review activity was highly concentrated among a relatively small share of games.
- Visibility and community activity were not distributed evenly across the catalog.



	app_name	total_reviews	approval_rate
1310	Call of Duty: Black Ops III	24543	57.11
3668	Heroes & Generals	31303	61.74
5340	No Man's Sky	24715	65.56
2033	DayZ	88850	66.23
5578	PAYDAY 2	88973	69.42
6427	Robocraft	41596	69.81
3408	Grand Theft Auto V	42374	74.00
6540	Rust	77037	79.22
4213	Killing Floor 2	21788	80.16
84	7 Days to Die	25070	80.39

Q3 Findings

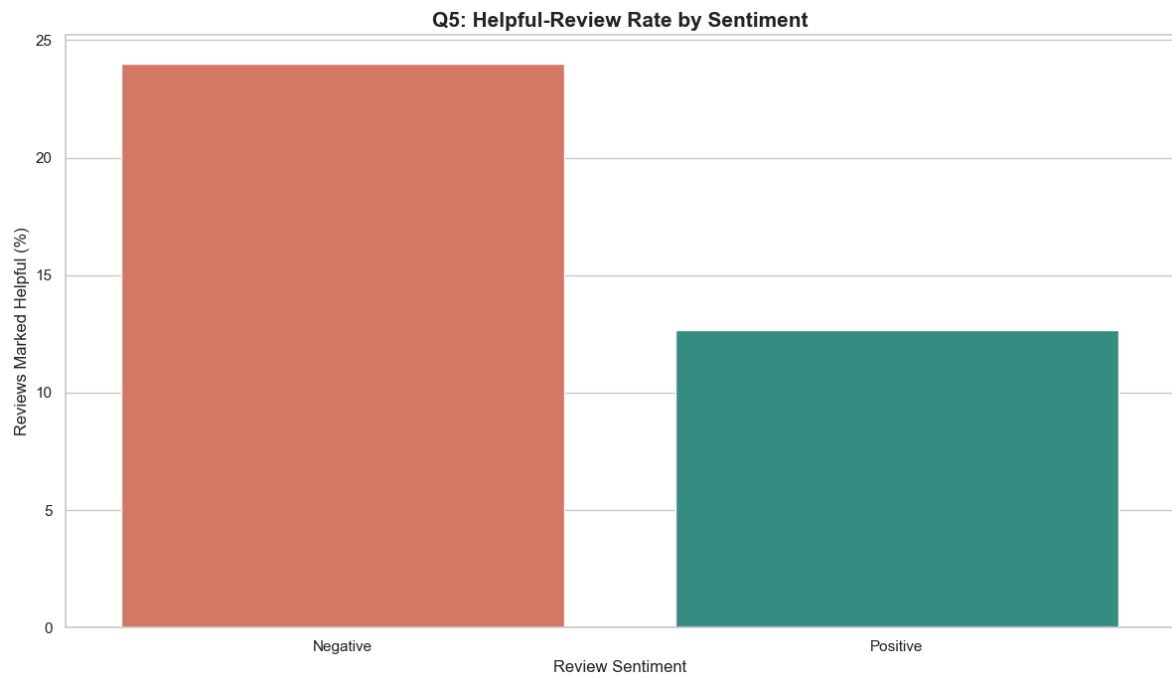
- These games recorded approval rates closest to a 50/50 split.
- The results indicate divided sentiment among games with more than 20,000 reviews.
- Review-text analysis would be required to determine the reasons behind the polarization.



	app_name	total_reviews	approval_rate	helpful_review_rate
5578	PAYDAY 2	88973	69.42	16.25
2033	DayZ	88850	66.23	16.10
6540	Rust	77037	79.22	13.38
3408	Grand Theft Auto V	42374	74.00	22.51
6427	Robocraft	41596	69.81	14.95
3668	Heroes & Generals	31303	61.74	15.78
5340	No Man's Sky	24715	65.56	24.84
1310	Call of Duty: Black Ops III	24543	57.11	19.87
8524	Trove	19031	74.98	12.34
2062	Dead by Daylight	18531	64.95	15.76

Q4 Findings

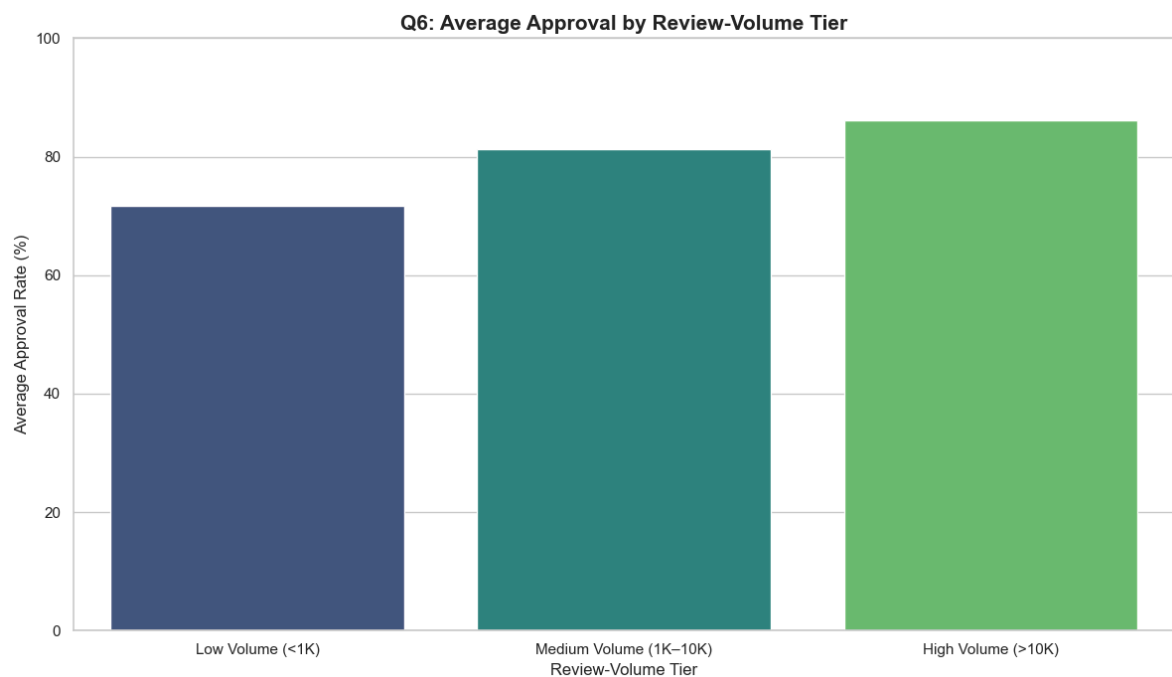
- **67 high-volume games** recorded approval rates below 80%.
- These games combined substantial review activity with comparatively weaker sentiment.
- They may be useful candidates for deeper product and review analysis.



	sentiment	total_reviews	helpful_review_rate
0	Negative	1101562	24.01
1	Positive	5132310	12.66

Q5 Findings

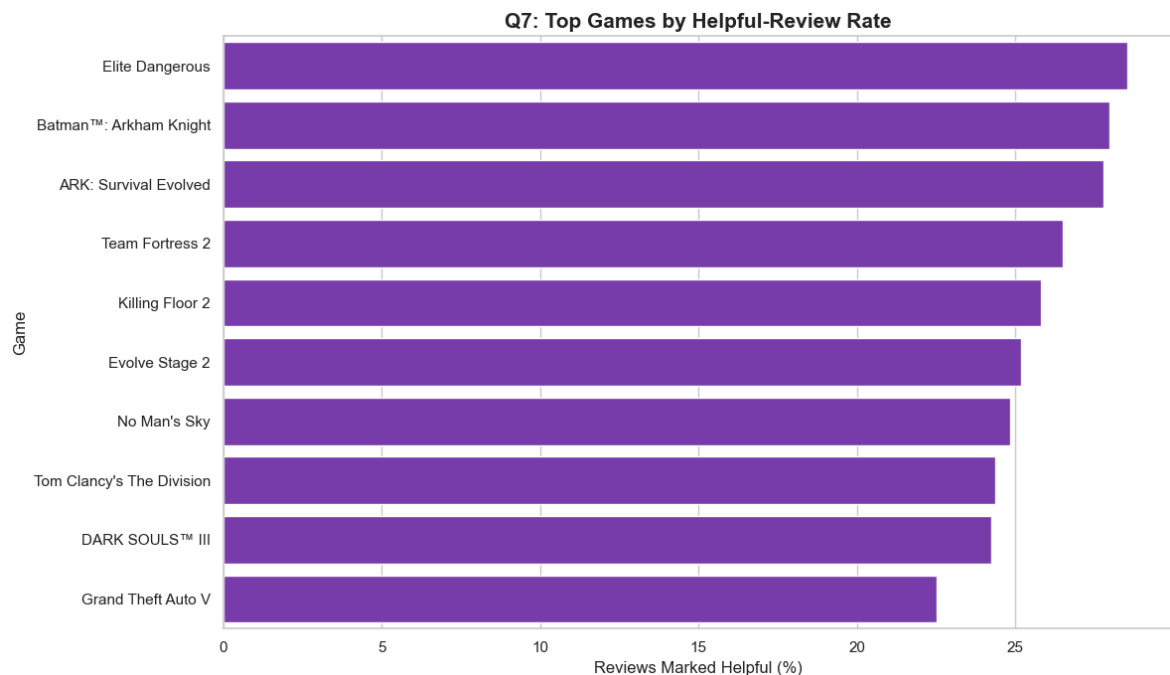
- **24.0% of negative reviews** were marked as Helpful.
- **12.7% of positive reviews** were marked as Helpful.
- Negative reviews were approximately **1.9 times more likely** to receive a Helpful mark.
- Critical feedback may therefore receive greater community visibility.



	review_volume_tier	game_count	average_approval
0	Low Volume (<1K)	8388	71.66
1	Medium Volume (1K–10K)	857	81.20
2	High Volume (>10K)	118	86.16

Q6 Findings

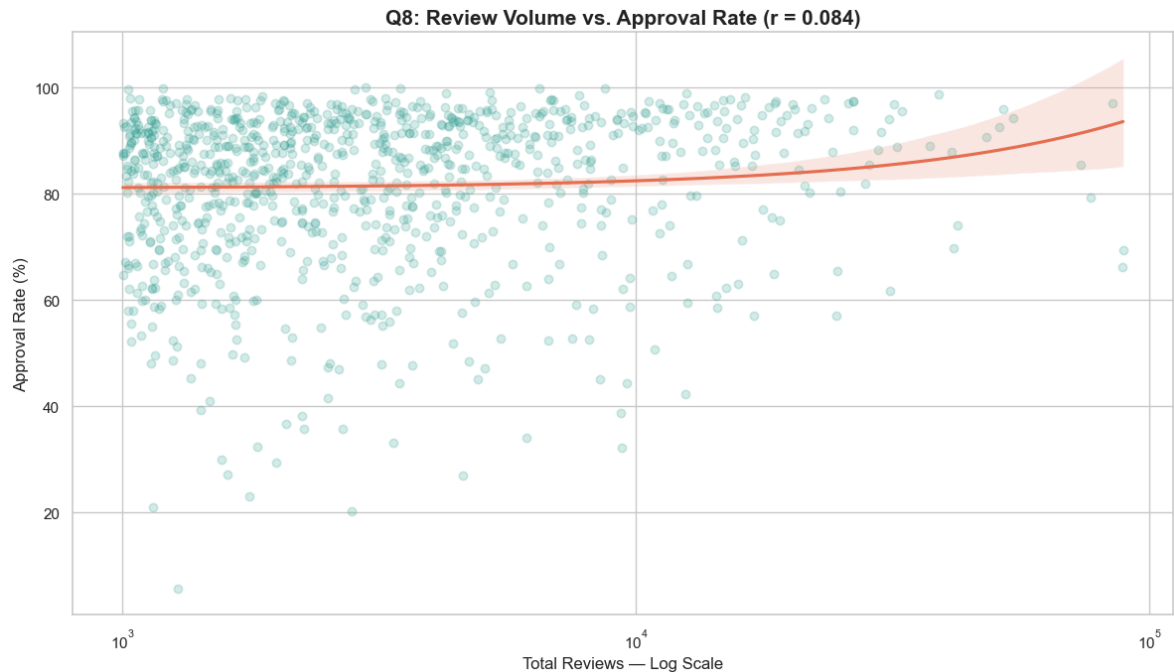
- Average approval increased from **71.7%** in the low-volume group to **86.2%** in the high-volume group.
- The difference was approximately **14.5 percentage points**.
- This is a descriptive relationship and does not prove causality.



	app_name	total_reviews	approval_rate	helpful_review_rate
2618	Elite Dangerous	12584	66.82	28.53
842	Batman™: Arkham Knight	16948	57.00	27.99
207	ARK: Survival Evolved	12470	42.30	27.79
7743	Team Fortress 2	12220	87.86	26.51
4213	Killing Floor 2	21788	80.16	25.82
2760	Evolve Stage 2	15827	63.03	25.18
5340	No Man's Sky	24715	65.56	24.84
8384	Tom Clancy's The Division	14309	60.86	24.36
1906	DARK SOULS™ III	16824	85.33	24.23
3408	Grand Theft Auto V	42374	74.00	22.51

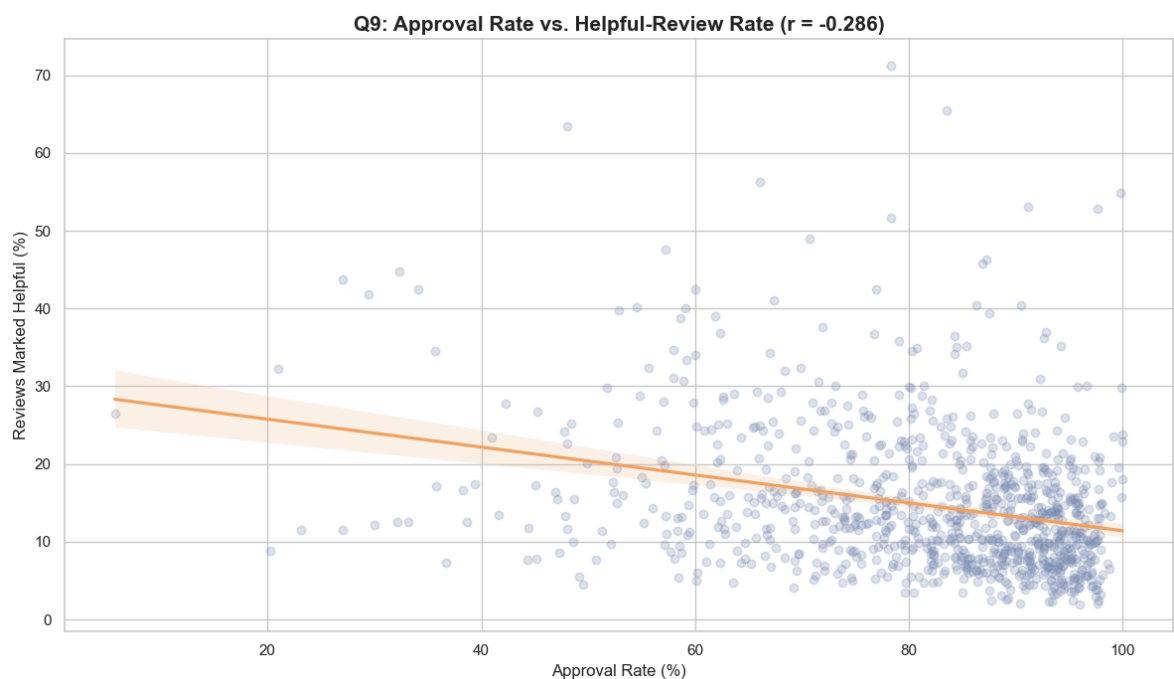
Q7 Findings

- **Elite Dangerous** had the highest helpful-review rate among games with more than 10,000 reviews.
- Approximately **28.5%** of its reviews were marked as Helpful.
- This indicates relatively strong interaction with its review content.



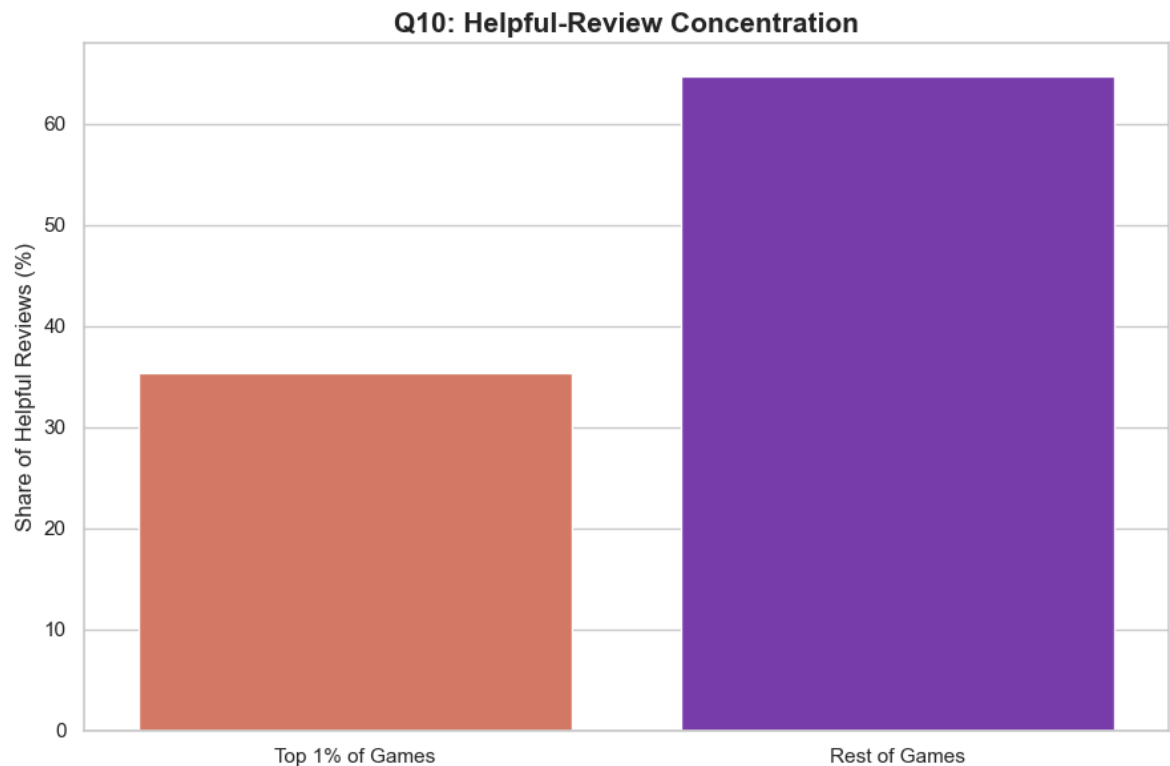
Q8 Findings

- Pearson correlation was **0.084**.
- The relationship between review volume and approval rate was very weak.
- Review volume alone is not a reliable indicator of user satisfaction.
- The logarithmic scale was used only to improve chart readability.



Q9 Findings

- Pearson correlation was **-0.286**.
- The result shows a weak negative relationship.
- Games with higher approval rates tended to have slightly lower helpful-review rates.
- This relationship does not prove causality.



	group	share
0	Top 1% of Games	35.30
1	Rest of Games	64.70

Q10 Findings

- The top **1% of games** accounted for **35.3% of all reviews marked as Helpful**.
- Helpful-review activity was highly concentrated among a small share of games.
- This result refers to review interaction, not sales or total player activity.

Executive Summary

- The original dataset contained **6,417,106 Steam reviews**.
- After cleaning, **6,233,872 valid reviews** remained.
- The reviews were aggregated into **9,363 game-level summaries**.
- Positive reviews represented **82.3%** of the dataset.
- **85.3%** of reviews were not marked as Helpful.
- The top **5% of games generated 69.7% of all reviews**.
- Negative reviews were approximately **1.9 times more likely** to receive a Helpful mark.
- Average approval increased by approximately **14.5 percentage points** between the low- and high-volume groups.
- Review volume had a correlation of **0.084** with approval rate.
- Approval rate had a correlation of **-0.286** with helpful-review rate.
- The top **1% of games accounted for 35.3% of all Helpful-marked reviews**.

Business Interpretation

Review activity and Helpful engagement were concentrated among a relatively small share of games.

Overall sentiment was positive, but review volume alone was not a reliable indicator of approval. Negative reviews were more likely to be marked as Helpful, suggesting that critical feedback may receive greater visibility.

The findings describe relationships within the dataset and do not prove causality.

```
=====
STEAM REVIEW ANALYSIS COMPLETED SUCCESSFULLY
=====
```

In []: